```
import sqlite3

import numpy as np
import pandas as pd
import streamlit as st
from scipy.stats import poisson

DB_PATH = "football_model.db"
DEFAULT_XG = 1.20
HOME_ADVANTAGE = 0.20
MAX_GOALS = 10


# ---------------------------------------------------------------- Βάση ----
def get_teams():
    """Επιστρέφει λίστα με όλες τις ομάδες της βάσης."""
    try:
        conn = sqlite3.connect(DB_PATH)
        rows = conn.execute(
            """
            SELECT home_team FROM matches
            UNION
            SELECT away_team FROM matches
            ORDER BY 1
            """
        ).fetchall()
        conn.close()
        return [r[0] for r in rows if r[0]]
    except sqlite3.Error:
        return []


def recent_avg(team, is_home, column, limit=5):
    """
    Μέσος όρος μιας στήλης (π.χ. home_xg) στα τελευταία `limit` παιχνίδια
    της ομάδας, εντός ή εκτός έδρας.
    Το LIMIT εφαρμόζεται σε υποερώτημα, ώστε να παίρνουμε πραγματικά τα
    τελευταία N παιχνίδια (στο αρχικό σου query το LIMIT δρούσε πάνω στο
    ήδη συγκεντρωμένο αποτέλεσμα, άρα δεν περιόριζε τίποτα).
    """
    team_col = "home_team" if is_home else "away_team"
    query = f"""
        SELECT AVG(val) FROM (
            SELECT {column} AS val FROM matches
```

```
            WHERE {team_col} = ?
            ORDER BY match_date DESC
            LIMIT ?
        )
    """
    try:
        conn = sqlite3.connect(DB_PATH)
        result = conn.execute(query, (team, limit)).fetchone()
        conn.close()
    except sqlite3.Error:
        return DEFAULT_XG
    return result[0] if result and result[0] is not None else DEFAULT_XG


# ---------------------------------------------------------- Μοντέλο ----
def predict(home_team, away_team, limit=5):
    # Επίθεση = xG που δημιουργεί η ομάδα
    home_attack = recent_avg(home_team, True, "home_xg", limit)
    away_attack = recent_avg(away_team, False, "away_xg", limit)
    # Άμυνα = xG που δέχεται η ομάδα
    home_defense = recent_avg(home_team, True, "away_xg", limit)
    away_defense = recent_avg(away_team, False, "home_xg", limit)

    home_xg = (home_attack + away_defense) / 2 + HOME_ADVANTAGE
    away_xg = (away_attack + home_defense) / 2

    goals = np.arange(MAX_GOALS + 1)
    matrix = np.outer(poisson.pmf(goals, home_xg), poisson.pmf(goals, away_xg))

    home_win = np.tril(matrix, -1).sum()
    draw = np.trace(matrix)
    away_win = np.triu(matrix, 1).sum()
    total = home_win + draw + away_win

    return {
        "home_xg": home_xg,
        "away_xg": away_xg,
        "home_win": home_win / total,
        "draw": draw / total,
        "away_win": away_win / total,
        "matrix": matrix / total,
    }


# ---------------------------------------------------------- UI -----
st.set_page_config(page_title="Πρόβλεψη Αγώνα", page_icon="⚽", layout="centered"
st.title("⚽ Πρόβλεψη Αγώνα (xG & Poisson)")
```

```
teams = get_teams()

with st.sidebar:
    st.header("Ρυθμίσεις")
    limit = st.slider("Τελευταία παιχνίδια για τον μέσο όρο", 3, 15, 5)
    st.caption(f"Βάση: `{DB_PATH}`")

if teams:
    col1, col2 = st.columns(2)
    home_team = col1.selectbox("Γηπεδούχος", teams, index=0)
    away_team = col2.selectbox("Φιλοξενούμενος", teams, index=min(1, len(teams) -
else:
    st.warning(
        "Δεν βρέθηκαν ομάδες στη βάση (ή δεν υπάρχει ο πίνακας `matches`). "
        "Μπορείς να πληκτρολογήσεις ομάδες χειροκίνητα· θα χρησιμοποιηθεί xG "
        f"{DEFAULT_XG:.2f} για κάθε ομάδα χωρίς δεδομένα."
    )
    col1, col2 = st.columns(2)
    home_team = col1.text_input("Γηπεδούχος").strip()
    away_team = col2.text_input("Φιλοξενούμενος").strip()

if st.button("Πρόβλεψη", type="primary", use_container_width=True):
    if not home_team or not away_team:
        st.error("Συμπλήρωσε και τις δύο ομάδες.")
    elif home_team == away_team:
        st.error("Η ομάδα γηπεδούχος και φιλοξενούμενος πρέπει να διαφέρουν.")
    else:
        r = predict(home_team, away_team, limit)

        st.subheader(f"{home_team} vs {away_team}")

        st.markdown("#### Εκτιμώμενα xG")
        c1, c2 = st.columns(2)
        c1.metric(home_team, f"{r['home_xg']:.2f}")
        c2.metric(away_team, f"{r['away_xg']:.2f}")

        st.markdown("#### Πιθανότητες (1X2) & δίκαιες αποδόσεις")
        df = pd.DataFrame(
            {
                "Σημείο": ["1", "X", "2"],
                "Αποτέλεσμα": [f"Νίκη {home_team}", "Ισοπαλία", f"Νίκη {away_team
                "Πιθανότητα": [
                    f"{r['home_win'] * 100:.1f}%",
                    f"{r['draw'] * 100:.1f}%",
                    f"{r['away_win'] * 100:.1f}%",
                ],
```

```
        "Δίκαιη απόδοση": [
            f"{1 / r['home_win']:.2f}",
            f"{1 / r['draw']:.2f}",
            f"{1 / r['away_win']:.2f}",
        ],
    }
)
st.dataframe(df, hide_index=True, use_container_width=True)

st.bar_chart(
    pd.DataFrame(
        {"Πιθανότητα": [r["home_win"], r["draw"], r["away_win"]]},
        index=["1", "X", "2"],
    )
)

# Πιο πιθανά ακριβή σκορ
m = r["matrix"]
top = sorted(
    ((i, j, m[i, j]) for i in range(6) for j in range(6)),
    key=lambda t: t[2],
    reverse=True,
)[:5]
with st.expander("Πιο πιθανά ακριβή σκορ"):
    st.table(
        pd.DataFrame(
            {
                "Σκορ": [f"{i}-{j}" for i, j, _ in top],
                "Πιθανότητα": [f"{p * 100:.1f}%" for _, _, p in top],
            }
        )
    )
```